Passo a passo para criação da primeira pipeline no GitHub Actions

01. Criar a pasta do projeto

Criar pasta chamada:

pipeline-cicd

02. Inicializa o projeto

Executar no terminal:

npm init -y

03. Criar soma.js dentro de src com o conteúdo:

```
function soma(a, b) {
  return a + b;
}
module.exports = soma;
```

04. Criar index.js com o conteúdo:

```
const soma = require('./src/soma');
console.log(soma(2, 3));
```

05. Dentro da pasta do projeto execute:

npm install jest --save-dev

06. Criar o arquivo de teste

tests/soma.test.js

Conteúdo:

```
const soma = require('../src/soma');
test('soma 2 + 3', () => {
  expect(soma(2, 3)).toBe(5);
});
```

07. Abra o package.json e adicione em scripts:

```
"scripts": {
  "test": "jest"
}
```

08. Agora basta rodar:

```
npm test
```

09. Criar arquivo .gitignore com o conteúdo:

```
node_modules/
.env
dist/
coverage/
```

10. Inicializar repositório

No terminal do VS Code, executar:

```
git init
```

11. Primeiro commit

No terminal do VS Code, executar:

```
git add .
git commit -m "Projeto inicial com testes"
```

Tudo ainda é local.

12. Criar repositório no GitHub

New repository

Nome: pipeline-demo

Criar.

13. Conectar remoto

No terminal do VS Code, executar conforme instrução dada ao criar o repositório. Exemplo:

git remote add origin https://github.com/SEUUSUARIO/pipeline-demo.git

git branch -M main

git push -u origin main

Agora o projeto está online.

14. Criar estrutura obrigatória

Dentro do projeto no VS Code:

Criar pasta:

.github

Dentro dela criar:

workflows

Estrutura final:

```
pipeline-demo/
 ├─ node_modules
 ├─ package-lock.json
 ├─ package.json
 ├─ index.js
└─ tests/
    └─ soma.test.js
└─ src/
    └─ soma.js
 └─ .github/
    └─ workflows/
```

15. Criar arquivo:

.github/workflows/pipeline.yml

Conteúdo do Pipeline:

```
name: CI Demo

on:
  push:
    branches: [ main ]

jobs:

  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout código
        uses: actions/checkout@v4

      - name: Instalar Node
        uses: actions/setup-node@v4
        with:
          node-version: 24

      - name: Instalar dependências
        run: npm install

      - name: Rodar testes
        run: npm test
```

16. Commit do pipeline

git add .

git commit -m "Adicionando pipeline CI/CD"

git push

Dando tudo certo deverá aparecer na aba Actions no GitHub, a execução efetuada com sucesso.

17. Adicionando Stage de Deploy

.github/workflows/pipeline.yml

Adicionar após ao stage de test.

```
  deploy:
    runs-on: ubuntu-latest
    needs: test

    steps:
      - name: Simular Deploy
        run: echo "Deploy realizado com sucesso!"
```

18. Gerando artefato no stage build

.github/workflows/pipeline.yml

```yaml
name: CI Demo

on:
  push:
    branches: [ main ]

jobs:

  build:

    runs-on: ubuntu-latest

    steps:
      - name: Checkout código
        uses: actions/checkout@v4

      - name: Instalar Node
        uses: actions/setup-node@v4
        with:
          node-version: 24

      - name: Instalar dependências
        run: npm install

      - name: Gerar artefato
        run: zip -r app.zip .

      - name: Publicar artefato
        uses: actions/upload-artifact@v4
        with:
```

```
      name: app-build
      path: app.zip

test:

  runs-on: ubuntu-latest
  needs: build

  steps:
    - name: Baixar artefato
      uses: actions/download-artifact@v4
      with:
        name: app-build

    - name: Extrair artefato
      run: unzip app.zip

    - name: Instalar Node
      uses: actions/setup-node@v4
      with:
        node-version: 24

    - name: Instalar dependências
      run: npm install

    - name: Rodar testes
      run: npm test

deploy:

  runs-on: ubuntu-latest
  needs: test
```

```
    steps:
      - name: Baixar artefato
        uses: actions/download-artifact@v4
        with:
          name: app-build

      - name: Simular deploy
        run: echo "Deploy realizado com sucesso!"
```

19. Separando em DEV e MAIN

Em local, criar a Branch DEV e executar um push por ela

Para criar Branch dev: *git checkout -b dev*

Criar environment de production no GitHub

name: CI - Development

```
on:
  push:
    branches: [ dev ]

  pull_request:
    branches: [ main ]

jobs:

  build:

    runs-on: ubuntu-latest
```

```
  steps:
    - name: Checkout código
      uses: actions/checkout@v4

    - name: Instalar Node
      uses: actions/setup-node@v4
      with:
        node-version: 24

    - name: Instalar dependências
      run: npm install

    - name: Gerar artefato
      run: zip -r app.zip .

    - name: Publicar artefato
      uses: actions/upload-artifact@v4
      with:
        name: app-build
        path: app.zip

test:

  runs-on: ubuntu-latest
  needs: build

  steps:
    - name: Baixar artefato
      uses: actions/download-artifact@v4
      with:
        name: app-build
```

```
      - name: Extrair artefato
        run: unzip app.zip

      - name: Instalar Node
        uses: actions/setup-node@v4
        with:
          node-version: 24

      - name: Instalar dependências
        run: npm install

      - name: Rodar testes
        run: npm test
```

name: CD - Production

```
on:
  push:
    branches: [ main ]

jobs:

  deploy:

    runs-on: ubuntu-latest
    environment: production

    steps:
      - uses: actions/checkout@v4

      - name: Simular deploy
```

run: echo "Deploy em produção realizado"

20. Utilizando o Lint

No projeto Node:

npm install eslint --save-dev

21. Inicializar configuração

npx eslint --init

Ele fará algumas perguntas. Para simplificar, pode escolher algo assim:

1. JavaScript
2. To check syntax and find problems
3. JavaScript modules (import/export)
4. None of these frameworks
5. No use TypeScript
6. Node
7. Yes, npm

Isso vai gerar o arquivo:

eslint.config.mjs

Nesse arquivo, substituir languageoptions por languageOptions: {globals:

{...globals.node, ...globals.jest }}

Abra o arquivo package.json e adicione:

```
  "scripts": {
    "lint": "eslint ."
  }
}
```

22. Testar localmente

npm run lint

23. Atualizar o workflow dev

```
name: CI - Development

on:
  push:
    branches: [ dev ]

  pull_request:
    branches: [ main ]

jobs:

  quality:

    runs-on: ubuntu-latest

    steps:
      - name: Checkout código
        uses: actions/checkout@v4

      - name: Instalar Node
        uses: actions/setup-node@v4
        with:
          node-version: 24

      - name: Instalar dependências
        run: npm install

      - name: Rodar lint
        run: npm run lint
```

```
build:

  runs-on: ubuntu-latest
  needs: quality

  steps:
    - name: Checkout código
      uses: actions/checkout@v4

    - name: Instalar Node
      uses: actions/setup-node@v4
      with:
        node-version: 24

    - name: Instalar dependências
      run: npm install

    - name: Gerar artefato
      run: zip -r app.zip .

    - name: Publicar artefato
      uses: actions/upload-artifact@v4
      with:
        name: app-build
        path: app.zip

test:

  runs-on: ubuntu-latest
  needs: build

  steps:
```

```
- name: Baixar artefato
  uses: actions/download-artifact@v4
  with:
    name: app-build

- name: Extrair artefato
  run: unzip app.zip

- name: Instalar Node
  uses: actions/setup-node@v4
  with:
    node-version: 24

- name: Instalar dependências
  run: npm install

- name: Rodar testes
  run: npm test
```

24. Utilizando Coverage

Rodar coverage com Jest no terminal:

npm test -- --coverage

Vai aparecer algo assim:

```
File        | % Stmts | % Branch | % Funcs | % Lines
------------|---------|----------|---------|--------
soma.js     |   100%  |   100%   |  100%   |  100%
```

- ***Stmts*** *→ linhas executadas*
- ***Funcs*** *→ funções testadas*
- ***Lines*** *→ cobertura geral*

25. Validando Coverage

Criar script no package.json

Para facilitar:

```
"scripts": {
  "test": "jest",
  "coverage": "jest --coverage"
}
```

Agora:

```
npm run coverage
```

26. Criar um cenário de baixa cobertura

Altere sua função:

```
function soma(a, b) {
  if (a === 0) {
    return b;
  }
  return a + b;
}
```

Coverage agora cai, por exemplo:

75%

27. Configurar limite mínimo de coverage

No package.json, adicione:

```
"jest": {
  "collectCoverage": true,
  "coverageThreshold": {
    "global": {
      "lines": 80
    }
  }
}
```

Se coverage < 80%:

❌ *Test suite failed due to low coverage*

28. Adicionar coverage no pipeline

No workflow:

```
- name: Run Coverage
  run: npm run coverage
```